

Fondamenti delle Reti Neurali

6.3.1 Dal Classificatore Lineare alle Reti Neurali

Un classificatore lineare computa gli score come:

$$s = Wx$$

Una **rete neurale** introduce la **non-linearità**:

$$s = W_2 \cdot f_{NL}(W_1 x)$$

Dove f_{NL} è una funzione non lineare (es. $\max(0, x)$).

□ **Attenzione:** Importanza della Non-Linearità

Senza la non-linearità, $s = W_2 \cdot W_1 x = Wx$ (prodotto di matrici = matrice), e ricadremmo nel caso del classificatore lineare!

6.3.1.1 Reti Multi-Layer

Si possono concatenare più layer:

$$s = W_3 \cdot f_{NL}(W_2 \cdot f_{NL}(W_1 x))$$

6.3.2 Il Neurone Artificiale

6.3.2.1 Modello Biologico

Il neurone artificiale è modellato sul **neurone biologico**:

- **Dendriti** → input multipli
- **Sinapsi** → pesi delle connessioni
- **Soma** → somma pesata + funzione di attivazione
- **Assone** → output singolo

6.3.2.2 Modello Matematico

$$y = f\left(\sum_i w_i x_i + b\right) = f(w^T x + b)$$

Dove:

- x_i = input
- w_i = pesi (weights)

- b = bias
- f = funzione di attivazione
- **Info:** Interpretazione

Un singolo neurone è un **classificatore binario** che divide lo spazio in due regioni.

6.3.3 Funzioni di Attivazione

6.3.3.1 Sigmoid

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- Output in $(0, 1)$
- **Problema:** saturazione per valori estremi $\rightarrow$ gradiente ≈ 0 (vanishing gradient)
- **Problema:** output non centrato in zero

6.3.3.2 Tanh

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

- Output in $(-1, 1)$
- Centrata in zero (meglio di sigmoid)
- **Problema:** satura ancora per valori estremi

6.3.3.3 ReLU (Rectified Linear Unit)

$$f(x) = \max(0, x)$$

- Computazionalmente **molto efficiente**
- Non satura per valori positivi
- Accelera la convergenza (6x rispetto a sigmoid/tanh)
- **Problema:** “neuroni morti” per input sempre negativi
- **Suggerimento:** Best Practice

ReLU è la scelta di default per le reti moderne. Dà in generale prestazioni migliori rispetto a sigmoid o tanh.

6.3.3.4 Leaky ReLU

$$f(x) = \begin{cases} x & \text{se } x > 0 \\ \alpha x & \text{se } x \leq 0 \end{cases}$$

Con α piccolo (es. 0.01). Risolve il problema dei neuroni morti.

6.3.4 Architettura di una Rete Neurale

6.3.4.1 Struttura a Layer

Una rete tipica è composta da **layer**:

- **Input Layer**: riceve i dati
- **Hidden Layers**: layer interni (da qui “deep learning”)
- **Output Layer**: produce la classificazione

□ **Info**: Convenzione

Il layer di input non viene contato. Una rete con 2 hidden layer + 1 output layer è detta “rete a 3 layer”.

6.3.4.2 Fully Connected Layer

In un layer **fully connected (FC)**, ogni neurone è connesso a **tutti** i neuroni del layer precedente.

6.3.4.3 Esempio: Rete a 3 Layer

Input (3 neuroni) → Hidden 1 (4 neuroni) → Hidden 2 (4 neuroni) → Output (1 neurone)

Con notazione matriciale:

$$h_1 = f(W_1x + b_1)$$

$$h_2 = f(W_2h_1 + b_2)$$

$$out = W_3h_2$$

Dove:

- W_1 : matrice 4×3
- W_2 : matrice 4×4
- W_3 : matrice 1×4

6.3.4.4 Representational Power

□ **Info**: Teorema (Approssimatore Universale)

Una rete con **almeno un layer nascosto** può approssimare con errore arbitrariamente piccolo qualsiasi funzione continua.

Tuttavia, reti più profonde sono spesso più **efficienti** per funzioni complesse.

6.3.5 Backpropagation

La **backpropagation** (retropropagazione dell'errore) è l'algoritmo fondamentale per addestrare le reti neurali. Permette di calcolare in modo efficiente il gradiente della loss rispetto a tutti i pesi della rete.

6.3.5.1 Idea Fondamentale

Per minimizzare la loss con il gradient descent, dobbiamo calcolare:

$$\frac{\partial L}{\partial w_{ij}}$$

per **tutti** i pesi della rete. La backpropagation usa la **regola della catena** per farlo in modo efficiente.

6.3.5.2 La Regola della Catena

Se $y = f(g(x))$, allora:

$$\frac{dy}{dx} = \frac{dy}{dg} \cdot \frac{dg}{dx}$$

6.3.5.3 Forward Pass e Backward Pass

Forward Pass:

1. I dati fluiscono dall'input verso l'output
2. Si calcola l'output di ogni layer
3. Si calcola la loss finale

Backward Pass:

1. Si calcola il gradiente della loss rispetto all'output
2. Si propaga il gradiente **all'indietro** layer per layer
3. Si aggiornano i pesi usando il gradiente

6.3.5.4 Esempio: Rete a 2 Layer

Consideriamo una rete semplice:

$$y = W_2 \cdot \text{ReLU}(W_1 x)$$
$$L = (y - t)^2$$

dove t è il target.

Forward Pass:

$$z_1 = W_1 x$$

$$a_1 = \text{ReLU}(z_1)$$

$$y = W_2 a_1$$

$$L = (y - t)^2$$

Backward Pass:

Partiamo dalla fine e risaliamo:

$$\frac{\partial L}{\partial y} = 2(y - t)$$

$$\frac{\partial L}{\partial W_2} = \frac{\partial L}{\partial y} \cdot a_1^T$$

$$\frac{\partial L}{\partial a_1} = W_2^T \cdot \frac{\partial L}{\partial y}$$

$$\frac{\partial L}{\partial z_1} = \frac{\partial L}{\partial a_1} \odot \text{ReLU}'(z_1)$$

$$\text{dove } \text{ReLU}'(z) = \begin{cases} 1 & \text{se } z > 0 \\ 0 & \text{altrimenti} \end{cases}$$

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z_1} \cdot x^T$$

6.3.5.5 Aggiornamento dei Pesi

Con il gradiente calcolato:

$$W := W - \eta \frac{\partial L}{\partial W}$$

dove η è il **learning rate**.

□ **Suggerimento:** Intuizione

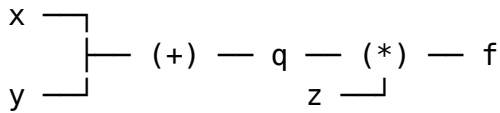
La backpropagation “distribuisce la colpa” dell’errore a ritroso attraverso la rete, permettendo a ogni peso di sapere quanto ha contribuito all’errore finale.

6.3.6 Computational Graph

Un modo utile per visualizzare la backpropagation è il **grafo computazionale**.

6.3.6.1 Esempio

Per $f(x, y, z) = (x + y) \cdot z$:



Forward: calcola $q = x + y$, poi $f = q \cdot z$

Backward:

- $\frac{\partial f}{\partial z} = q$
- $\frac{\partial f}{\partial q} = z$
- $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \cdot \frac{\partial q}{\partial x} = z \cdot 1 = z$
- $\frac{\partial f}{\partial y} = z \cdot 1 = z$

6.3.6.2 Gradienti Locali

Ogni nodo del grafo:

1. Riceve il gradiente "upstream" $\frac{\partial L}{\partial \text{output}}$
 2. Calcola il gradiente locale $\frac{\partial \text{output}}{\partial \text{input}}$
 3. Propaga il prodotto ai nodi precedenti
-

6.3.7 Problemi della Backpropagation

6.3.7.1 Vanishing Gradient

Con funzioni come sigmoid/tanh, il gradiente può diventare **molto piccolo** nei layer iniziali:

- La derivata di sigmoid è al massimo 0.25
- Moltiplicando molti valori < 1 , il gradiente "svanisce"

Soluzioni:

- Usare ReLU
- Inizializzazione appropriata dei pesi
- Batch normalization
- Skip connections (ResNet)

6.3.7.2 Exploding Gradient

Il gradiente può anche **esplodere** se i pesi sono troppo grandi.

Soluzioni:

- Gradient clipping
- Inizializzazione appropriata
- Batch normalization

6.3.8 Inizializzazione dei Pesi

Una buona inizializzazione è **cruciale** per la convergenza.

6.3.8.1 Regole Pratiche

Attivazione	Inizializzazione Consigliata
ReLU	He initialization: $w \sim \mathcal{N}(0, \sqrt{2/n_{in}})$
Sigmoid/Tanh	Xavier initialization: $w \sim \mathcal{N}(0, \sqrt{1/n_{in}})$

□ **Attenzione:** Mai Inizializzare a Zero!

Se tutti i pesi sono zero, tutti i neuroni calcolano la stessa cosa e il gradiente è uguale per tutti → simmetria che impedisce l'apprendimento.

I **bias** sono normalmente inizializzati a **zero**.

6.3.9 Learning Rate

Il **learning rate** η è uno degli iperparametri più critici.

6.3.9.1 Effetti del Learning Rate

Learning Rate	Effetto
Troppo alto	Divergenza, oscillazioni, convergenza a minimi locali subottimali
Troppo basso	Convergenza molto lenta, possibile blocco in minimi locali
Giusto	Convergenza rapida e stabile

6.3.9.2 Learning Rate Scheduling

È comune **ridurre** il learning rate durante l'addestramento:

- **Step decay**: riduci di un fattore ogni N epoche
 - **Exponential decay**: $\eta_t = \eta_0 \cdot e^{-\lambda t}$
 - **1/t decay**: $\eta_t = \eta_0 / (1 + \lambda t)$
-

6.3.10 Reti Ricorrenti vs Feed-Forward

Tipo	Caratteristica
Feed-Forward	Nessun ciclo, informazione fluisce solo in avanti
Ricorrente (RNN)	Presenti cicli, memoria degli input precedenti

Le **reti ricorrenti** sono usate per sequenze (testo, serie temporali) ma non sono trattate in questo corso.
